

BUILD_VARIANTS

Build Variants Documentation

Overview

Rhythm uses Android build variants (product flavors) to create different builds for different distribution channels while maintaining a single codebase.

Quick Reference

```
# Build commands
./gradlew assembleFdroidRelease      # For F-Droid
./gradlew assembleGithubRelease      # For GitHub Releases

# List all build variants
./gradlew tasks --all | grep assemble
```

Build Configuration

Flavor Dimension: distribution

All flavors belong to the distribution dimension, ensuring they're mutually exclusive.

Product Flavors

1. fdroid

Purpose: F-Droid repository distribution

Package: chromahub.rhythm.app

Version Suffix: -fdroid

BuildConfig Flags:

```
ENABLE_YOUTUBE_MUSIC = true
ENABLE_SPOTIFY_CANVAS = true
ENABLE_APPLE_MUSIC = true
ENABLE_DEEZER = true
ENABLE_LRCLIB = true
ENABLE_SPOTIFY_SEARCH = true
```

Default Settings: APIs and auto-updates disabled by default (opt-in for F-Droid compliance)

Philosophy: FOSS users expect maximum functionality. F-Droid isn't bound by corporate app store policies, but opt-in defaults ensure compliance.

2. github

Purpose: Direct GitHub releases

Package: chromahub.rhythm.app

Version Suffix: -gh

BuildConfig Flags:

```
ENABLE_YOUTUBE_MUSIC = true
ENABLE_SPOTIFY_CANVAS = true
ENABLE_APPLE_MUSIC = true
ENABLE_DEEZER = true
ENABLE_LRCLIB = true
ENABLE_SPOTIFY_SEARCH = true
```

Default Settings: APIs and auto-updates enabled by default (opt-out available)

Use Case: Users who download directly from GitHub releases get the full-featured experience.

How BuildConfig Flags Work

1. Compile-Time Conditional Compilation

Flags are defined in app/build.gradle.kts:

```
buildConfigField("boolean", "ENABLE_YOUTUBE_MUSIC", "true")
```

2. Runtime API Initialization

APIs are initialized conditionally in `NetworkClient.kt`:

```
val ytmusicApiService: YTMusicApiService? = if
    (BuildConfig.ENABLE_YOUTUBE_MUSIC) {
    ytmusicRetrofit.create(YTMusicApiService::class.java)
} else null
```

3. Settings UI Adaptation

UI components check `BuildConfig` before rendering:

```
if (BuildConfig.ENABLE_YOUTUBE_MUSIC) {
    ApiServiceRow(title = "YouTube Music", ...)
}
```

4. Runtime Feature Checks

Code checks both `BuildConfig` AND user settings:

```
fun isYTMusicApiEnabled(): Boolean =
    BuildConfig.ENABLE_YOUTUBE_MUSIC &&
    (appSettings?.ytMusicApiEnabled?.value ?: false)
```

Default Settings by Variant:

- **F-Droid:** APIs and auto-updates disabled by default (opt-in for compliance)
- **GitHub:** APIs and auto-updates enabled by default (opt-out available)

This ensures:

- Build variants properly control feature availability
 - Users can toggle features on/off within allowed limits
 - F-Droid compliance through opt-in defaults
 - Single codebase maintains all variants
-

Variant-Specific Resources

Directory Structure

```
app/src/
├── main/                # Shared code and resources
├── fdroid/
│   └── res/
│       └── values/
│           └── config.xml
```

```
└─ github/
    └─ res/
        └─ values/
            └─ config.xml
```

Resource Flags

Each variant has a config.xml with boolean resources:

fdroid/res/values/config.xml:

```
<bool name="feature_youtube_music_available">true</bool>
<bool name="feature_spotify_canvas_available">true</bool>
<string name="distribution_channel">F-Droid</string>
```

github/res/values/config.xml:

```
<bool name="feature_youtube_music_available">true</bool>
<bool name="feature_spotify_canvas_available">true</bool>
<string name="distribution_channel">GitHub</string>
```

Build Output

APK Naming Convention

Rhythm-{versionName}-{suffix}-{flavor}-{buildType}.apk

Examples:

Rhythm-4.0.312.858-fdroid-fdroid-release.apk

Rhythm-4.0.312.858-gh-github-release.apk

Output Locations

APKs:

app/build/outputs/apk/fdroid/release/

app/build/outputs/apk/github/release/

App Bundles (AAB):

app/build/outputs/bundle/fdroidRelease/app-fdroid-release.aab

app/build/outputs/bundle/githubRelease/app-github-release.aab

Testing Variants

Build All Variants

```
# Release builds
./gradlew assembleRelease

# Debug builds
./gradlew assembleDebug

# Specific variant
./gradlew assembleFdroidDebug
```

Install Specific Variant

```
# Install F-Droid debug
./gradlew installFdroidDebug

# Install GitHub release
adb install app/build/outputs/apk/github/release/Rhythm-*.apk
```

Verify BuildConfig Values

After building, check generated BuildConfig:

F-Droid:

```
app/build/generated/source/buildConfig/fdroid/release/chromahub/
rhythm/app/BuildConfig.java
```

Should contain:

```
public static final boolean ENABLE_YOUTUBE_MUSIC = true;
public static final boolean ENABLE_SPOTIFY_CANVAS = true;
```

GitHub:

```
app/build/generated/source/buildConfig/github/release/chromahub/
rhythm/app/BuildConfig.java
```

Should contain:

```
public static final boolean ENABLE_YOUTUBE_MUSIC = true;
public static final boolean ENABLE_SPOTIFY_CANVAS = true;
```

CI/CD Integration

GitHub Actions Example

```
name: Build All Variants

on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'

      - name: Build F-Droid variant
        run: ./gradlew assembleFdroidRelease

      - name: Build GitHub variant
        run: ./gradlew assembleGithubRelease

      - name: Upload artifacts
        uses: actions/upload-artifact@v3
        with:
          name: apk-variants
          path: app/build/outputs/apk/**/release/*.apk
```

F-Droid Metadata

Add to .fdroid.yml or metadata/*.yml:

```
Builds:
- versionName: 4.0.312.858
  versionCode: 40312858
  gradle:
    - fdroid

# F-Droid will automatically use the fdroid flavor
```

Troubleshooting

Issue: Wrong variant built

Symptom: APK has unexpected features enabled/disabled

Solution: Verify you ran the correct assemble task (e.g.,
./gradlew assembleFdroidRelease)

Issue: UI shows disabled features

Symptom: F-Droid build hides YouTube Music settings

Solution: Check BuildConfig.ENABLE_YOUTUBE_MUSIC in generated files

Issue: Null pointer when calling API

Symptom: Crash when accessing ytmusicApiService

Solution: Ensure null-safe access:

```
if (ytmusicApiService != null) {  
    ytmusicApiService.search(...)  
}
```

Issue: Build fails with variant not found

Symptom: Task 'assembleFdroid' not found

Solution: Use full variant name: assembleFdroidRelease or
assembleFdroidDebug

Maintenance

Adding a New Build Variant

1. Add flavor in app/build.gradle.kts:

```
create("newvariant") {  
    dimension = "distribution"  
    applicationId = "chromahub.rhythm.app"  
    buildConfigField("boolean", "ENABLE_YOUTUBE_MUSIC", "true")  
    versionNameSuffix = "-new"  
}
```

1. Create variant directory:

```
mkdir -p app/src/newvariant/res/values
```

1. Add config.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="distribution_channel">New Variant</string>
</resources>
```

1. Test build:

```
./gradlew assembleNewvariantRelease
```

Removing a Build Variant

1. Delete flavor block from build.gradle.kts
 2. Remove app/src/variantname/ directory
 3. Sync Gradle: ./gradlew --refresh-dependencies
-

Best Practices

1. Version Consistency

- All variants share the same versionCode and versionName
- Only versionNameSuffix differs for logging

2. Feature Parity

- F-Droid and GitHub should always have identical features

3. Testing

- Test each variant separately before release
- Ensure disabled features don't appear in UI
- Verify API calls are actually blocked

4. Documentation

- Keep this README updated when adding/removing features
 - Document any variant-specific behavior
-

FAQ

Q: Can I install multiple variants on the same device?

A: No, they share the same applicationId. Use debug builds with different IDs for testing.

Q: Which variant should I use for local development?

A: Use fdroid or github debug builds for full feature access during development.

Q: How do I know which variant is installed?

A: Check Settings → About → Build Info or check version suffix in logs.

Q: Can users switch between variants?

A: No, they must uninstall and reinstall a different variant. Data won't transfer.

Q: Why not use separate apps with different package names?

A: Single package name simplifies distribution across different channels.

Related Documentation

- [Build Instructions](#)
- [Contributing Guide](#)

Last Updated: January 19, 2026
Rhythm Music Player Build System Documentation